TiffinBox

Complete Project Plan & Implementation Doc

1. Project Overview

Problem Definition

Many students and working professionals living away from home depend on mess food or restaurants for their daily meals. These options often provide limited variety, repetitive taste, and sometimes unhealthy food. At the same time, many homemakers prepare fresh meals daily for their families, but their cooking capacity could easily support a few additional portions.

However, there is no simple platform that connects people who want home-cooked food with nearby home cooks who are willing to prepare a few extra meals.

One Line Description

A city-based platform where home cooks post their daily menu and nearby customers pre-order fresh homemade meals before lunch or dinner cutoff time.

Problems Solved

User	Problem	Solution
Customer	Zomato/Swiggy is expensive (Rs.150-200/meal)	Home cook charges Rs.60-80, no middleman
Customer	No homemade food option	Real home cooks making fresh regional food
Cook	Food wastage - cook extra just in case	Pre-order cutoff means cook knows exact quantity
Cook	Uncertain daily income	Dashboard shows confirmed orders & earnings
Cook	No platform to reach customers	Profile page with photos, ratings, cuisine type
Both	Awkward cash collection	Razorpay handles payments digitally

Core Innovation

The 2-3 hour pre-order window is what makes this different from everything else:

- Swiggy/Zomato = order anytime, reheated food
- TiffinBox = pre-order fresh, cook makes it just for you
- Cook never wastes food — knows exact quantity to cook
- Customer gets fresh homemade food — not reheated

2. Users & Flows

3 User Roles

Role	What They Do
Customer	Browses cooks, pre-orders meals, pays online, tracks order
Home Cook	Posts daily menu, accepts orders, arranges delivery via Porter/Dunzo
Admin	Approves cooks, monitors platform, views stats

Customer Flow

- Register / Login → Select city
- Browse verified home cooks in same city
- Click on a cook → See profile + today's menu
- Select dish + quantity → Check cutoff time
- Place order → Enter delivery address → Pay via Razorpay
- Order confirmed → Track live status (Confirmed → Preparing → Ready → Delivered)
- Receive meal → Rate & review the cook

Home Cook Flow

- Register → Fill profile (bio, cuisine, city, photo)
- Wait for Admin approval
- Once approved → Access cook dashboard
- Every morning → Post today's menu (dish name, stock photo, price, max portions, cutoff time)
- Customers start ordering → See incoming orders in real-time
- Accept or reject each order
- Update order status → Preparing → Ready → Delivered
- End of day → View earnings summary

Admin Flow

- Login to admin panel
- See pending cook registrations → Review → Approve or Reject
- Monitor all active orders on platform
- Handle complaints or disputes
- View platform analytics (total users, orders, active cooks)
- See all the data of Cook and Customer (User, Order, Menu, Rating etc.)

3. Full Feature List

Auth System

- Register & login with email/password
- Role selection at register — Customer or Cook
- Email verification via OTP
- JWT authentication with protected routes per role
- Forgot password via email OTP

Customer Features

- Select city at registration
- Browse verified home cooks in same city
- View cook profile — bio, cuisine, rating, reviews
- View today's menu — dish name, photo, price, portions left, cutoff time
- Place order before cutoff time
- Enter delivery address at checkout
- Pay via Razorpay test mode
- View order status — Confirmed → Preparing → Ready → Delivered
- Order history
- Rate & review cook after delivery

Home Cook Features

- Create profile — name, bio, city, cuisine type, photo
- Wait for admin approval before going live
- Post today's menu every morning (dish name, price, max portions, cutoff time, stock photo)
- See incoming orders with customer name & delivery address
- Accept or reject orders
- Update order status
- View simple earnings summary

Admin Features

- Login to separate admin panel
- View pending cook registrations → Approve or Reject
- View all users, cooks, orders
- Basic platform stats — total users, orders, active cooks

Notifications

- Email via Nodemailer: order placed (to cook), order confirmed (to customer), order delivered
- In-app notification bell (basic)

Review System

- Star rating (1-5) + optional written review
- Only customers who ordered (and received) can review
- Cook's average rating shown on profile

Building In Next Version (1.1)

Feature	Reason Deferred
Google OAuth	Keep auth simple for now
Real-time Socket.io	Page polling sufficient for MVP
Live delivery tracking	Cook uses Porter/Dunzo independently
In-app chat	Email notifications sufficient
Subscription plans	Pay-per-meal is simpler
Payout system	Manual payouts for MVP
Dispute management	Admin handles manually

4. Tech Stack

Technology Stack

Layer	Technology
Frontend	React + Redux Toolkit + Tailwind CSS
Backend	Node.js + Express.js
Database	MongoDB + Mongoose
Authentication	JWT (JSON Web Tokens)
Payments	Razorpay (test mode)
Image Upload	Cloudinary
Email	Nodemailer (Gmail SMTP)
Frontend Deploy	Vercel
Backend Deploy	Render
Database Host	MongoDB Atlas

5. Database Schema

Collections Overview

Collection	Purpose
Users	All users — customer, cook, admin
CookProfiles	Extended cook info, ratings, earnings
Menus	Daily menu posted by each cook
Orders	All orders placed by customers
Reviews	Ratings & reviews for cooks
Notifications	In-app notifications for all users

1. Users Collection

```
Users {
  name            String, required
  email           String, required, unique
  password        String, required (bcrypt hashed)
  role            Enum['customer', 'cook', 'admin']
  city            String, required
  phone           String
  avatar          String (Cloudinary URL)
  isActive        Boolean, default: true
  resetOTP        String
  resetOTPExpiry  Date
  createdAt       Date
}
```

2. CookProfiles Collection

```
CookProfiles {
  userId          ref -> Users, unique
  bio             String
  cuisineType     [String]  e.g. ['Gujarati', 'Punjabi']
  city            String
  address         String
  photo           String (Cloudinary URL)
  isVerified      Boolean, default: false
  isAvailable     Boolean, default: true
  rating          Number, default: 0
  totalReviews    Number, default: 0
  earnings: {
    total         Number, default: 0
    thisMonth     Number, default: 0
    thisWeek      Number, default: 0
  }
  createdAt       Date
}
```

3. Menus Collection

```
Menus {
  cookId          ref -> CookProfiles
  date            Date (today's date)
  mealType        Enum['lunch', 'dinner']
  dishes: [{
    name          String
    photo         String (stock photo URL)
    price         Number
    maxPortions   Number
    portionsLeft  Number
    description   String (optional)
  }]
  cutoffTime      String  e.g. '10:00'
  isActive        Boolean, default: true
  createdAt       Date
}
```

4. Orders Collection

```
Orders {
  customerId        ref -> Users
  cookId            ref -> CookProfiles
  menuId            ref -> Menus
  dish: {
    name            String
    photo           String
    price           Number
  }
  quantity          Number
  totalAmount       Number
  deliveryAddress   String, required
  status            Enum[
                      'pending', 'confirmed',
                      'preparing', 'ready',
                      'delivered', 'cancelled'
                    ]
  paymentStatus     Enum['pending', 'paid', 'failed']
  paymentId         String (Razorpay payment ID)
  isReviewed        Boolean, default: false
  createdAt         Date
}
```

5. Reviews Collection

```
Reviews {
  customerId      ref -> Users
  cookId          ref -> CookProfiles
  orderId         ref -> Orders, unique
  rating          Number (1-5), required
  comment         String (optional)
  createdAt       Date
}
```

6. Notifications Collection

```
Notifications {
  userId          ref -> Users
  message         String
  type            Enum[
                    'order_placed', 'order_confirmed',
                    'order_preparing', 'order_delivered',
                    'order_cancelled', 'cook_approved'
                  ]
  isRead          Boolean, default: false
  createdAt       Date
}
```

6. API Endpoints

Auth Routes

Method	Endpoint	Description	Auth
POST	/api/auth/register	Register new user	Public
POST	/api/auth/login	Login user	Public
POST	/api/auth/forgot-password	Send OTP to email	Public
POST	/api/auth/reset-password	Reset password with OTP	Public
GET	/api/auth/me	Get logged in user info	Private
POST	/api/auth/verify-otp	Email OTP Verification	Private

Cook Routes

Method	Endpoint	Description	Auth
POST	/api/cook/profile	Create cook profile	Cook
GET	/api/cook/profile	Get own cook profile	Cook
PUT	/api/cook/profile	Update cook profile	Cook
GET	/api/cook/all?city=	Get all verified cooks by city	Customer
GET	/api/cook/:id	Get single cook profile (public)	Public
PUT	/api/cook/availability	Toggle available/unavailable	Cook

Menu Routes

Method	Endpoint	Description	Auth
POST	/api/menu	Cook posts today's menu	Cook
GET	/api/menu/today/:cookId	Get today's menu of a cook	Public
GET	/api/menu/my	Cook sees their own menu	Cook
PUT	/api/menu/:id	Update menu	Cook
DELETE	/api/menu/:id	Delete menu	Cook
GET	/api/menu/all	Get All menu	Cook
GET	/api/menu/history	Get all expired menu	Cook

Order Routes

Method	Endpoint	Description	Auth
POST	/api/orders	Customer places order	Customer
GET	/api/orders/my	Customer's order history	Customer
GET	/api/orders/cook	Cook's incoming orders	Cook
PUT	/api/orders/:id/status	Cook updates order status	Cook
PUT	/api/orders/:id/cancel	Customer cancels order	Customer
POST	/api/orders/verify-payment	Verify Razorpay payment	Customer

Review Routes

Method	Endpoint	Description	Auth
POST	/api/reviews	Customer submits review	Customer
GET	/api/reviews/cook/:cookId	Get all reviews of a cook	Public

Notification Routes

Method	Endpoint	Description	Auth
GET	/api/notifications	Get all notifications for user	Private
PUT	/api/notifications/read	Mark all as read	Private

Admin Routes

Method	Endpoint	Description	Auth
GET	/api/admin/cooks/pending	Get unverified cooks	Admin
PUT	/api/admin/cooks/:id/verify	Approve cook	Admin
PUT	/api/admin/cooks/:id/reject	Reject cook	Admin
GET	/api/admin/users	Get all users	Admin
GET	/api/admin/orders	Get all orders	Admin
GET	/api/admin/stats	Platform stats	Admin
PUT	/api/admin/users/:id/ban	Ban a user	Admin
GET	/api/admin/users/:id	Single User Detailed Data	Admin

7. Key Business Logic

Logic	Where to Implement	How
Check cutoff time before order	Order controller	Compare current time with menu cutoffTime — reject if passed
Decrease portionsLeft on order	Order controller	MongoDB findByIdAndUpdate with $inc: { portionsLeft: -quantity }
Auto close menu when portions = 0	Order controller	After decrement, if portionsLeft <= 0, set isActive: false
Only delivered orders can be reviewed	Review controller	Check order status === 'delivered' before saving review
Only verified cooks shown	Cook controller	Filter by isVerified: true in all customer-facing queries
Update cook rating after review	Review controller	Recalculate average rating and update CookProfile
Send email on order status change	Order controller	Call sendEmail() utility on every status update
Create notification on order event	Order controller	Create Notification document for both cook and customer
City based discovery	Cook controller	Filter cooks by city === customer's city
Order can be cancelled in only Preparing Stage	Order controller	Check order status == preparing,